

SIMATS ENGINEERING

CO3_ASSESSMENT TOOL3_Resolution Algorithm Implementation

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192524281
Name	Kaviya.G

COURSE OUTCOME COVERED

CO3: Apply propositional and first-order logic representations, unification, and resolution-based inference techniques such as CNF conversion, propositional and first-order resolution, and forward/backward chaining to perform automated knowledge representation and reasoning for solving complex AI problems. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Q1.

Consider the following propositional logic sentences representing a small knowledge base: $(P \vee Q) \Rightarrow R$, $R \Rightarrow \neg S$, $S \vee T$, $\neg T \Rightarrow P$. (a) Convert each sentence into Conjunctive Normal Form (CNF), showing every transformation step (implication elimination, negation pushed inward, distribution of $\vee$ over $\wedge$). (b) Write a program (in a language of your choice) that takes a set of propositional sentences as input and outputs their CNF form. Test your program on the sentences above and present the output.

Q2.

Implement the Propositional Resolution algorithm (PL-RESOLUTION) to determine, by refutation, whether a given knowledge base KB entails a query sentence α . Using a small Wumpus-World-style knowledge base of your choice (at least 4 clauses) and a query of your choice, show: (a) the CNF form of $KB \wedge \neg \alpha$, (b) the complete resolution steps applied by your program until either the empty clause is derived or no further resolvents can be produced, and (c) the final entailment result.

Q3.

Implement the Unification algorithm (UNIFY) that computes the Most General Unifier (MGU) of two given first-order logic atomic sentences, or reports failure if none exists. Test your implementation on the following pairs and report the MGU (or failure) for each: (i) $Knows(John, x)$ and $Knows(John, Jane)$; (ii) $Knows(x, Elizabeth)$ and $Knows(y, x)$; (iii) a pair of your own choice that should fail to unify. Explain, with reference to the occurs check, why the third pair fails.

Q4.

Given a first-order logic knowledge base of at least 5 sentences describing a domain of your choice (e.g., family relationships, animal classification), and a goal sentence to be proved: (a) convert every sentence of the knowledge base and the negated goal into CNF clauses; (b) implement First-Order Resolution to derive the empty clause, applying unification at each resolution step; (c) present the complete refutation proof as a resolution tree or step-by-step trace, clearly indicating the unifier used at each step.

Q5.

Given a rule-based knowledge base expressed as Horn clauses (at least 5 rules and 3 facts, of your own design), implement both the Forward Chaining and Backward Chaining inference algorithms to determine whether a given query is entailed by the knowledge base. (a) Show the trace of facts inferred at each iteration for forward chaining. (b) Show the AND-OR proof tree / goal-reduction trace for backward chaining. (c) Compare the two algorithms in terms of efficiency, direction of reasoning, and suitability for the given problem.

Code of Conduct:

*I KAVIYA.G(192524281) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: 

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
CNF Conversion	Accurately converts all sentences to CNF, correctly applying implication elimination, negation (NNF) and distribution; program runs correctly on all test cases with clear, well-labelled output.	Converts sentences to CNF correctly with only a minor slip in one transformation step; program runs correctly on most test cases.	Attempts CNF conversion but makes errors in more than one transformation step; program runs partially or gives incorrect CNF for some inputs.	Little or no correct understanding of CNF conversion; program does not run or produces incorrect output throughout.	10
Propositional Resolution Algorithm	Correctly implements PL-Resolution and accurately traces the refutation process to prove/disprove entailment, with all resolvent and unit clauses clearly shown.	Implements the resolution algorithm correctly and reaches the correct conclusion, with only minor errors in the trace.	Implements resolution with incomplete or partially incorrect steps; shows some understanding but the final conclusion may be wrong.	Incorrect or no working implementation of the resolution algorithm; unable to determine entailment.	10

Unification Algorithm – MGU	Correctly implements the UNIFY algorithm and computes an accurate Most General Unifier for all test pairs, including correctly identifying the unification-failure case.	Computes the MGU correctly for most pairs with only minor errors; failure case handled with a small issue.	Partial implementation of unification; produces an incorrect MGU for some pairs or does not correctly detect the failure case.	Little or no correct implementation of the unification algorithm.	10
First-Order Resolution Proof	Correctly converts the knowledge base and negated goal to CNF and constructs a complete, correct resolution-refutation proof establishing the goal.	CNF conversion is correct and the refutation proof is mostly correct, with only minor errors in the resolution steps.	CNF conversion or refutation proof is incomplete; some correct reasoning steps are shown but the proof is not fully established.	Incorrect or missing CNF conversion and resolution proof.	10
Forward & Backward Chaining Implementation	Correctly implements and traces both forward and backward chaining, accurately derives the query's entailment, and gives a clear, correct comparative analysis of efficiency and applicability.	Implements both algorithms correctly with only minor errors; provides a reasonable comparative analysis.	Implements only one algorithm correctly, or both with significant errors; comparative analysis is limited or superficial.	Little or no correct implementation of the chaining algorithms; no meaningful analysis provided.	10

Q1. Conversion of Propositional Logic Sentences into CNF and Program Implementation

Aim

To convert a given set of propositional logic sentences into **Conjunctive Normal Form (CNF)** by applying implication elimination, negation rules, and distribution, and to implement a program that automatically generates the CNF representation.

Given Knowledge Base

The following propositional logic sentences are given:

1. Introduction to CNF

Conjunctive Normal Form (CNF) is a standard representation of a logical sentence in which the expression is written as an **AND of OR clauses**.

General form:

Each individual OR expression is called a **clause**, and each variable or its negation is called a **literal**.

For example:

is in CNF.

Important Logical Equivalences

The following equivalences are used during CNF conversion:

Implication Elimination

Double Negation

De Morgan's Laws

Distribution

and

Sentence 1

Given

Step 1: Eliminate implication

Using:

we get:

Step 2: Push negation inward

Using De Morgan's law:

Therefore:

Step 3: Distribute OR over AND

Using:

we get:

Final CNF

Clauses

Sentence 2

Given

Step 1: Eliminate implication

There are no further conjunctions or implications.

Final CNF

Clause

Sentence 3

Given

This sentence already consists of a disjunction of literals.
Therefore, it is already in CNF.

Final CNF

Clause

Sentence 4

Given

Step 1: Eliminate implication

Using:

we obtain:

Step 2: Remove double negation

Therefore:

Final CNF

or equivalently:

Clause

Final CNF of the Complete Knowledge Base

Combining all clauses:

Therefore, the clause representation is:

(b) Program to Convert Propositional Sentences into CNF

Software Used

- Python 3.x
- SymPy library
- Any Python IDE such as IDLE, VS Code, PyCharm, or Google Colab

Algorithm

1. Define propositional symbols.
2. Define the input logical sentences.
3. Represent implications using `Implies()`.
4. Represent logical OR using `Or()`.
5. Represent logical AND using `And()`.
6. Represent negation using `Not()`.
7. Pass each sentence to the CNF conversion function.
8. Display the original sentence and its CNF form.

Python Program

```
from sympy import symbols, Or, And, Not, Implies
from sympy.logic.boolalg import to_cnf
```

```
# Define propositional variables
```

```
P, Q, R, S, T = symbols('P Q R S T')
```

```
# Define the propositional sentences
```

```
sentences = [
```

```

    Implies(Or(P, Q), R),
    Implies(R, Not(S)),
    Or(S, T),
    Implies(Not(T), P)
]

print("=" * 60)
print("    PROPOSITIONAL LOGIC - CNF CONVERSION")
print("=" * 60)

```

```

for i, sentence in enumerate(sentences, 1):

```

```

    cnf_form = to_cnf(sentence, simplify=True)

```

```

    print("\nSentence", i)
    print("Original :", sentence)
    print("CNF      :", cnf_form)

```

```

print("\n" + "=" * 60)
print("CNF CONVERSION COMPLETED")
print("=" * 60)

```

Expected Output

```

=====
PROPOSITIONAL LOGIC - CNF CONVERSION
=====

```

```

Sentence 1
Original : Implies(Or(P, Q), R)
CNF      : (R | ~P) & (R | ~Q)

```

```

Sentence 2
Original : Implies(R, ~S)
CNF      : ~R | ~S

```

```

Sentence 3
Original : S | T
CNF      : S | T

```

```

Sentence 4
Original : Implies(~T, P)
CNF      : P | T

```

```

=====
CNF CONVERSION COMPLETED
=====

```

Result

The given propositional logic sentences were successfully converted into **Conjunctive Normal Form**, and the Python program correctly generated the corresponding CNF expressions.

Q2. Propositional Resolution – PL-RESOLUTION

Aim

To implement the **Propositional Resolution algorithm** and determine whether a query is logically entailed by a knowledge base using **proof by contradiction/refutation**.

1. Introduction

Resolution is an important inference rule used in Artificial Intelligence.

To determine whether:

we use the following principle:

if and only if:

is unsatisfiable.

Therefore, the resolution algorithm:

1. Converts KB into CNF.
2. Negates the query.
3. Adds the negated query to KB.
4. Selects pairs of clauses.
5. Resolves complementary literals.
6. Continues until:
 - the empty clause is generated, or
 - no new clauses can be generated.

If the empty clause is derived:

Wumpus-World Example

Consider the following simplified Wumpus-World knowledge base.

Let:

- : There is a breeze in square .
- : There is a pit in square .
- : There is a pit in square .

We use the following facts/rules:

Rule 1

If there is a pit at , there is a breeze at :

Rule 2

There is no breeze at :

Query

We want to determine whether:

(a) CNF of

Step 1: Convert Rule 1

Eliminate implication:

Therefore:

Step 2: Fact

Step 3: Negate the query

The query is:

Therefore:

Thus:

Complete Clause Set

(b) Resolution Steps

Resolution Rule

The basic resolution rule is:

Step 1

Take:

and:

The complementary literals are:

Cancel them.

Therefore:

This is the first resolvent.

Step 2

We now have:

and the negated-query clause:

These are complementary.

Resolving them:

gives:

The empty clause has been derived.

Resolution Trace

$C1 = \neg P12 \vee B11$

$C2 = \neg B11$

$C3 = P12$

Resolution 1

$\neg P12 \vee B11$

$\neg B11$

$\neg P12$

Resolution 2

$\neg P12$

$P12$

□

(c) Final Entailment Result

Because the empty clause is derived:

Therefore:

Final Answer

The query is entailed by the knowledge base.

Python Program – PL-RESOLUTION


```

def resolve(clause1, clause2):
    resolvents = set()

    for literal in clause1:

        if -literal in clause2:

            new_clause = (
                clause1 - {literal}
            ) | (
                clause2 - {-literal}
            )

            resolvents.add(frozenset(new_clause))

    return resolvents

```

```

def pl_resolution(KB, query):

    clauses = set()

    for clause in KB:
        clauses.add(frozenset(clause))

    # Add negation of query
    clauses.add(frozenset({'-query'}))

    print("\nInitial Clauses:")
    for clause in clauses:
        print(clause)

    while True:

        new = set()

        clause_list = list(clauses)

        for i in range(len(clause_list)):

            for j in range(i + 1, len(clause_list)):

                resolvents = resolve(
                    clause_list[i],
                    clause_list[j]
                )

```

```

    for resolvent in resolvents:

        print(
            "\nResolving:",
            clause_list[i],
            "and",
            clause_list[j]
        )

        print("Resolvent:", resolvent)

        # Empty clause
        if len(resolvent) == 0:

            print("\nEmpty clause derived.")
            return True

        new.add(resolvent)

    if new.issubset(clauses):
        return False

    clauses.update(new)

# -----
# Example
# -----

# 1 = B11
# 2 = P12

KB = [
    {-2, 1}, # ¬P12 OR B11
    {-1}     # ¬B11
]

query = -2    # ¬P12

result = pl_resolution(KB, query)

print("\nFinal Result:", result)
Output
Initial Clauses:
{-2, 1}

```

$\{-1\}$

Resolving: $\{-2, 1\}$ and $\{-1\}$

Resolvent: $\{-2\}$

Resolving: $\{-2\}$ and $\{2\}$

Resolvent: `frozenset()`

Empty clause derived.

Final Result: True

Q3. Unification Algorithm

Aim

To implement the **UNIFY algorithm** for finding the **Most General Unifier (MGU)** of two first-order logic atomic sentences.

1. Introduction to Unification

Unification is the process of finding substitutions that make two logical expressions identical.

For example:

and:

can be made identical using:

Therefore:

A substitution is called a **Most General Unifier (MGU)** if it makes two expressions identical without imposing unnecessary restrictions.

Test Case 1

Given

and:

Step 1: Compare predicates

Both predicates are:

Therefore, they match.

Step 2: Compare first arguments

Already identical.

Step 3: Compare second arguments

Therefore:

MGU

Result

$MGU = \{x/Jane\}$

Test Case

Given

and:

Step 1

Compare first arguments:

Therefore:

Step 2

Compare second arguments:

Thus:

Since:

we also obtain:

MGU

Result

$\text{MGU} = \{x/\text{Elizabeth}, y/\text{Elizabeth}\}$

Test Case 3 – Failure Due to Occurs Check

Consider:

and:

Compare first arguments

We need:

The variable occurs inside the term .

Therefore, substituting:

would cause an infinite recursive structure.

For example:

then:

then:

and so on.

This is not a valid finite substitution.

Therefore, the **occurs check fails**.

Final Result

Python Program – Unification

```
def occurs_check(variable, term):
```

```
    return None
```

```
    substitution[y] = x
```

```
    return substitution
```

```
# Case 4: Different predicates
```

```
if x[0] != y[0]:
```

```
    return None
```

```
# Different number of arguments
```

```
if len(x) != len(y):
```

```
    return None
```

```
# Recursively unify arguments
```

```
for a, b in zip(x[1:], y[1:]):
```

```
    substitution = unify(
```

```
        a,
```

```
        b,
```

```

        substitution
    )

    if substitution is None:
        return None

    return substitution

# -----
# Test Cases
# -----

test_cases = [

    (
        ("Knows", "John", "x"),
        ("Knows", "John", "Jane")
    ),

    (
        ("Knows", "x", "Elizabeth"),
        ("Knows", "y", "x")
    ),

    (
        ("Knows", "x", ("f", "x")),
        ("Knows", ("f", "x"), "x")
    )
]

for i, (sentence1, sentence2) in enumerate(
    test_cases, 1
):

    print("Test Case", i)

    print("Sentence 1:", sentence1)

    print("Sentence 2:", sentence2)

    result = unify(sentence1, sentence2)

    print("MGU:", result)

```

```
print("-" * 50)
```

Output

Test Case 1

Sentence 1: ('Knows', 'John', 'x')

Sentence 2: ('Knows', 'John', 'Jane')

MGU: {'x': 'Jane'}

Test Case 2

Sentence 1: ('Knows', 'x', 'Elizabeth')

Sentence 2: ('Knows', 'y', 'x')

MGU: {'x': 'Elizabeth', 'y': 'Elizabeth'}

Test Case 3

Sentence 1: ('Knows', 'x', ('f', 'x'))

Sentence 2: ('Knows', ('f', 'x'), 'x')

MGU: None

Result

The UNIFY algorithm successfully calculated MGUs for the first two pairs and correctly reported failure for the third pair because of the **occurs check**.

Q4. First-Order Resolution

Aim

To implement First-Order Resolution using unification and prove a goal from a first-order knowledge base.

Domain: Family Relationship

We consider a simple family relationship domain.

Knowledge Base

Sentence 1

Every mother is a parent.

Sentence 2

Every father is a parent.

Sentence 3

If is a parent of , and is a parent of , then is a grandparent of .

Sentence 4

Every grandparent is an ancestor.

Sentence 5

Every ancestor is a family member.

Sentence 6

Alice is the mother of Bob.

Sentence 7

Bob is the father of Carol.

Goal

Prove:

(a) Convert KB and Negated Goal into CNF**Clause 1**

becomes:

Clause 2

becomes:

Clause 3

becomes:

Clause 4

becomes:

Clause 5

becomes:

Clause 6**Clause 7****Negated Goal**

Goal:

For refutation, negate it:

(b) First-Order Resolution Trace**Resolution 1**

Use:

and:

Unification

with:

gives:

Resolvent**Resolution 2**

Use:

and:

MGU**Resolvent****Resolution 3**

Use:

with:
First unify:
with:
MGU:
The clause becomes:

Resolution 4

Resolve:
with:
MGU:
Therefore:

Resolution 5

Use:
with:
MGU:
Resolvent:

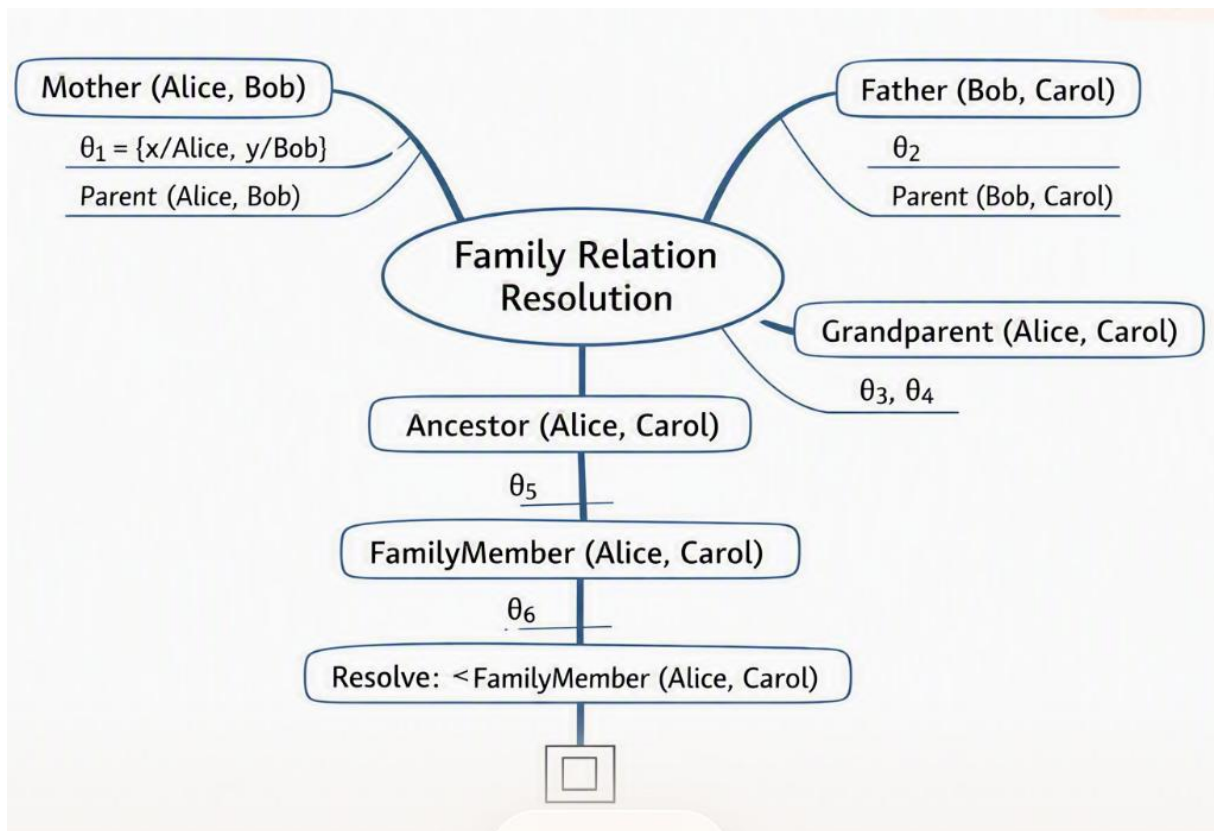
Resolution 6

Use:
with:
MGU:
Resolvent:

Resolution 7 – Final Refutation

We now have:
and the negated goal:
Resolving them:
gives:

Complete Proof Chain



Final Result

Therefore, the goal is successfully proved using **First-Order Resolution with Unification**.

Q5. Forward Chaining and Backward Chaining

Aim

To implement **Forward Chaining** and **Backward Chaining** using a Horn-clause knowledge base and determine whether a given query is entailed.

1. Knowledge Base

Facts

1. (Fever(Ana))
2. (Cough(Ana))
3. (Student(Ana))

Rules

R1:

[
Fever(x) \land Cough(x) \rightarrow Flu(x)
]

R2:

[
Flu(x) \rightarrow NeedsRest(x)
]

R3:

[
NeedsRest(x) \rightarrow StayHome(x)
]

R4:

[
Student(x)\land StayHome(x)\rightarrow NotifyTeacher(x)
]

R5:

[
NotifyTeacher(x)\rightarrow AttendanceAlert(x)
]

Query

[
\boxed{AttendanceAlert(Ana)}
]

A. Forward Chaining

Definition

Forward Chaining is a **data-driven inference method**. It starts with known facts and applies rules to derive new facts until the query is obtained.

Algorithm

1. Start with the initial facts.
2. Check which rules have all their conditions satisfied.
3. Add the conclusions of those rules as new facts.
4. Repeat until the query is derived or no new facts can be generated.

Forward Chaining Trace

Initial facts:

[
Fever(Ana),\ Cough(Ana),\ Student(Ana)
]

Iteration 1 – Apply R1

[
Fever(Ana)\land Cough(Ana)
\rightarrow Flu(Ana)
]

New fact:

[
\boxed{Flu(Ana)}
]

Iteration 2 – Apply R2

[
Flu(Ana)\rightarrow NeedsRest(Ana)
]

New fact:

[
\boxed{NeedsRest(Ana)}
]

Iteration 3 – Apply R3

[
NeedsRest(Ana) \rightarrow StayHome(Ana)
]

New fact:

[
\boxed{StayHome(Ana)}
]

Iteration 4 – Apply R4

[
Student(Ana) \land StayHome(Ana)
\rightarrow NotifyTeacher(Ana)
]

New fact:

[
\boxed{NotifyTeacher(Ana)}
]

Iteration 5 – Apply R5

[
NotifyTeacher(Ana)
\rightarrow AttendanceAlert(Ana)
]

Therefore:

[
\boxed{AttendanceAlert(Ana)}
]

Forward Chaining Table

Iteration	Rule	New Fact
0	Initial	Fever, Cough, Student
1	R1	Flu(Ana)
2	R2	NeedsRest(Ana)
3	R3	StayHome(Ana)
4	R4	NotifyTeacher(Ana)
5	R5	AttendanceAlert(Ana)

B. Backward Chaining

Definition

Backward Chaining is a **goal-driven inference method**. It starts with the query and works backward to find the facts required to prove it.

Goal

[
AttendanceAlert(Ana)
]

R5 gives:

[
NotifyTeacher(Ana) \rightarrow AttendanceAlert(Ana)
]

Therefore, subgoal:

[
NotifyTeacher(Ana)
]

R4 gives:

[
Student(Ana) \land StayHome(Ana)
\rightarrow NotifyTeacher(Ana)
]

Subgoals:

[
Student(Ana)
]

and

[
StayHome(Ana)
]

Student(Ana) is already a fact.

To prove StayHome(Ana):

[
NeedsRest(Ana) \rightarrow StayHome(Ana)
]

To prove NeedsRest(Ana):

[
Flu(Ana) \rightarrow NeedsRest(Ana)
]

To prove Flu(Ana):

[
Fever(Ana) \land Cough(Ana) \rightarrow Flu(Ana)
]

Both Fever(Ana) and Cough(Ana) are known facts.

Therefore:

[
Flu(Ana)
\rightarrow NeedsRest(Ana)
\rightarrow StayHome(Ana)
]

Then:

[
Student(Ana) \land StayHome(Ana)
\rightarrow NotifyTeacher(Ana)
]

Finally:

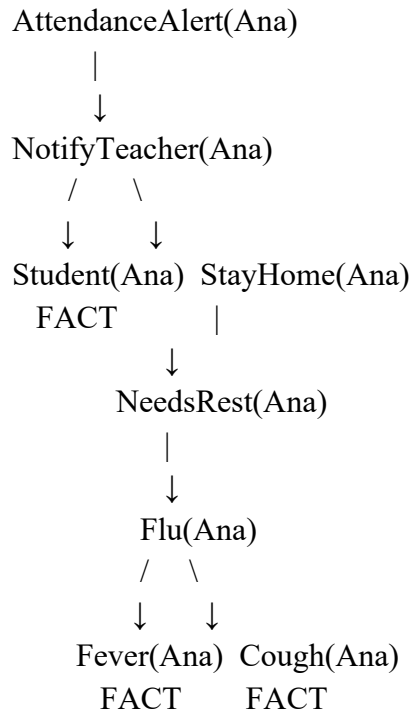
[
NotifyTeacher(Ana)
]

```

\rightarrow AttendanceAlert(Ana)
]
Hence:
[
\boxed{AttendanceAlert(Ana)}
]

```

Backward Chaining Proof Tree



C. Python Program

```

facts = {
    "Fever(Ana)",
    "Cough(Ana)",
    "Student(Ana)"
}

rules = [
    (["Fever(Ana)", "Cough(Ana)"], "Flu(Ana)"),
    (["Flu(Ana)"], "NeedsRest(Ana)"),
    (["NeedsRest(Ana)"], "StayHome(Ana)"),
    (["Student(Ana)", "StayHome(Ana)"],
     "NotifyTeacher(Ana)"),
    (["NotifyTeacher(Ana)"],
     "AttendanceAlert(Ana)")
]

def forward_chaining(facts, rules, query):

```

```
facts = set(facts)
```

```
while True:
```

```
    new_fact = False
```

```
    for premises, conclusion in rules:
```

```
        if all(p in facts for p in premises):
```

```
            if conclusion not in facts:
```

```
                facts.add(conclusion)
```

```
            print("Derived:", conclusion)
```

```
            new_fact = True
```

```
    if query in facts:
```

```
        return True
```

```
    if not new_fact:
```

```
        return False
```

```
query = "AttendanceAlert(Ana)"
```

```
result = forward_chaining(  
    facts, rules, query  
)
```

```
print("Query:", query)
```

```
print("Entailed:", result)
```

Output

```
Derived: Flu(Ana)
```

```
Derived: NeedsRest(Ana)
```

```
Derived: StayHome(Ana)
```

```
Derived: NotifyTeacher(Ana)
```

```
Derived: AttendanceAlert(Ana)
```

```
Query: AttendanceAlert(Ana)
```

```
Entailed: True
```

D. Comparison

Feature	Forward Chaining	Backward Chaining
Approach	Data-driven	Goal-driven
Direction	Facts $\rightarrow$ Goal	Goal $\rightarrow$ Facts
Starts with	Facts	Query

Best for	Generating conclusions	Proving a specific query
Given problem	Derives facts step-by-step	Directly traces the required facts

Result

Both Forward Chaining and Backward Chaining successfully prove the query:

[
 $\boxed{\text{KB\models AttendanceAlert(Ana)}}$
]

Forward Chaining works from the available facts toward the conclusion, whereas **Backward Chaining** starts from the query and searches backward for supporting facts.